# Software Requirements Specification (SRS): Smart Attendance System

## 1. Introduction

### 1.1. Purpose

This document provides a detailed specification of the requirements for the Smart Attendance System. It is intended for the development team to understand the system's functionalities, constraints, and goals.

### 1.2. Scope

The software will facilitate the automated capturing, storing, managing, and reporting of student attendance. It will be a **web-based application** for administrators, instructors, students, and parents. The system will be designed to support **RFID or Biometric** attendance marking technologies.

### 1.3. Definitions & Acronyms

- **SRS:** Software Requirements Specification
- **SIS:** Student Information System
- **UI:** User Interface
- **API:** Application Programming Interface
- **RFID:** Radio-Frequency Identification

## 2. Overall Description

### 2.1. Product Perspective

The Smart Attendance System will be a standalone system that can also be integrated with existing university/school management software. It will replace manual attendance tracking methods, providing a centralized and automated solution.

### 2.2. User Characteristics

- **Instructors:** Tech-savvy enough to use web applications but require an intuitive and simple UI.
- **Administrators:** Trained personnel who will manage the system's backend data and configurations.
- **Students:** Familiar with mobile and web technology.
- **Parents:** Varying levels of technical expertise; notifications should be simple and direct (e.g., SMS).

### 2.3. System Architecture (High-Level)

The system will follow a client-server architecture with the following technology stack:

- **Frontend (Client):** A responsive web application built with **React.js and TypeScript**.
- **Backend (Server):** A server-side application built with **Node.js and Express.js, using TypeScript**. The backend will follow a modular architecture with separate directories for routes, services, controllers, etc.
- **Database:** The system will use **Supabase** for its PostgreSQL database, authentication, and other backend services.
- **Hardware Interface:** A module to interface with the chosen attendance capture hardware (**RFID reader or Biometric scanner**).

# 3. Functional Requirements

## 3.1. User Management

- **FR1.1:** The system shall allow administrators to create, view, update, and deactivate user accounts (students, instructors, parents).
- **FR1.2:** The system shall require all users to log in with a unique username and password.
- **FR1.3:** The system shall implement role-based access control.

## 3.2. Attendance Management

- **FR2.1:** The system shall record attendance with a timestamp when a student scans their **RFID card or provides a biometric scan (e.g., fingerprint)**.
- **FR2.2:** The system shall automatically mark students as 'Present', 'Absent', or 'Late' based on pre-defined class timings.
- **FR2.3:** The system shall allow instructors to manually modify an attendance record for a specific student with a reason.
- **FR2.4:** The system shall display real-time attendance status to the instructor on their dashboard.

## 3.3. Reporting

- **FR3.1:** The system shall generate attendance reports for a specific class, student, or date range.
- **FR3.2:** Reports shall be downloadable in PDF and CSV formats.
- **FR3.3:** The system shall provide graphical representations of attendance data (e.g., charts, graphs).

## 3.4. Notifications

- **FR4.1:** The system shall send automated email/SMS notifications to registered parents/guardians when a student is marked 'Absent'.

# 4. Non-Functional Requirements

### 4.1. Performance

- **NFR1.1:** The system shall mark a student's attendance within 2 seconds of a successful scan.
- **NFR1.2:** The web interface shall load key pages (e.g., dashboard, reports) within 3 seconds.
- **NFR1.3:** The system shall support concurrent usage by up to 50 instructors and 1000 students during peak hours.

### 4.2. Security

- **NFR2.1:** All user passwords must be hashed and salted.
- **NFR2.2:** The system must be protected against common web vulnerabilities (e.g., SQL Injection, XSS).
- **NFR2.3:** All data transmission between the client and server shall be encrypted using TLS/SSL.

### 4.3. Usability

- **NFR3.1:** The user interface shall be intuitive and require minimal training for new users.
- **NFR3.2:** The system shall be responsive and accessible on standard web browsers (Chrome, Firefox, Safari) and mobile devices.

### 4.4. Reliability

- **NFR4.1:** The system shall have an uptime of 99.5%.
- **NFR4.2:** The system must have a backup and recovery mechanism to prevent data loss.

## 5. Interface Requirements

### 5.1. Hardware Interfaces

- The system will need to interface with **RFID readers or Biometric scanners**. This will require a specific driver or SDK for the chosen hardware, likely communicating via USB or a TCP/IP connection.

### 5.2. Software Interfaces

- **SIS/LMS Integration (Optional):** An API will be developed to allow for data synchronization (student lists, course schedules) with existing institutional systems.

## 6. System Design Details

### 6.1. User Interface (UI) Description

The UI will be a clean, modern, and responsive web application.

- **Common Pages:**
  - **Login Page:** A single, standard email and password form for all user types. After

successful login, users will be redirected to their respective dashboards based on their role.
    - **Reports Page:** A common interface for both **Admins and Instructors** to generate and view reports, with options to export to PDF/CSV. Admins will have a broader view (all courses), while instructors will be limited to their assigned courses.
- **Admin Dashboard:**
    - **Main Dashboard:** Sidebar navigation (User Management, Course Management, Reports) and a content area with key metrics and charts.
    - **User/Course Management:** Tables to display data with forms in modals for creating/editing entries.
- **Instructor Dashboard:**
    - **Main Dashboard:** A list of the instructor's daily courses with real-time attendance counts.
    - **Class View:** A grid view of students in a class, showing their attendance status. Allows for manual overrides.
- **Student/Parent View:**
    - **Dashboard:** A profile page showing a calendar view of attendance and a summary of attendance percentages per course.

## 6.2. API Endpoints (RESTful)

| Method | Endpoint | Description | Access Role |
|---|---|---|---|
| POST | /api/auth/login | Authenticate a user and return a JWT token. | All |
| GET | /api/users | Get a list of all users. | Admin |
| POST | /api/users | Create a new user. | Admin |
| PUT | /api/users/:id | Update a user's details. | Admin |
| GET | /api/courses | Get a list of all courses. | Admin |
| POST | /api/courses | Create a new course. | Admin |
| GET | /api/attendance/live/:courseId | Get real-time attendance for a | Instructor |

| | | | |
|---|---|---|---|
| | | class. | |
| POST | /api/attendance/record | Record attendance from a hardware scan. | System |
| PUT | /api/attendance/override | Manually override a student's attendance. | Instructor |
| GET | /api/reports | Generate an attendance report. | **Admin, Instructor** |
| GET | /api/student/attendance | Get attendance history for the logged-in student or their child. | Student, Parent |

## 6.3. Database Schema (Supabase - PostgreSQL)

**users table**

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | UUID | PRIMARY KEY | Unique identifier for the user. |
| email | VARCHAR(255) | UNIQUE, NOT NULL | User's email address. |
| full_name | VARCHAR(255) | NOT NULL | User's full name. |
| role | ENUM | NOT NULL ('admin', 'instructor', 'student', 'parent') | The role of the user. |
| parent_id | UUID | FOREIGN KEY (references users.id), NULL | The parent associated with a student account. |

**courses table**

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | UUID | PRIMARY KEY | Unique identifier for the course. |
| course_name | VARCHAR(255) | NOT NULL | The name of the course. |
| course_code | VARCHAR(50) | UNIQUE, NOT NULL | The unique code for the course. |
| instructor_id | UUID | FOREIGN KEY (references users.id) | The instructor for the course. |

**enrollments table**

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | UUID | PRIMARY KEY | Unique identifier for enrollment. |
| student_id | UUID | FOREIGN KEY (references users.id) | The enrolled student. |
| course_id | UUID | FOREIGN KEY (references courses.id) | The course they are enrolled in. |

**attendance_records table**

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | UUID | PRIMARY KEY | Unique identifier for the record. |
| student_id | UUID | FOREIGN KEY (references users.id) | The student being marked. |

| | | | |
|---|---|---|---|
| course_id | UUID | FOREIGN KEY (references courses.id) | The course for the attendance. |
| timestamp | TIMESTAMPTZ | default: now() | The exact time of the scan. |
| status | ENUM | NOT NULL ('Present', 'Absent', 'Late') | The calculated status. |
| is_override | BOOLEAN | default: false | True if manually changed. |
| override_reason | TEXT | NULL | Reason for manual override. |